

LAB IMPLEMENTATION 6

Creation and Deployment of a Ballot Smart Contract

BC

Aim

To create, compile, deploy, and interact with a Ballot smart contract using Solidity and Remix IDE.
To perform voting and delegation transactions and observe the contract state through Remix.

Theory

A Solidity smart contract is a program deployed on a blockchain whose state and functions can be accessed through transactions and calls. The Ballot contract models voting with a `Voter` structure, a `Proposal` structure, an address-based mapping, and a dynamic array of proposals. The constructor initializes the chairperson, gives the chairperson voting weight, and stores the supplied proposal names.

The contract uses `require` statements to enforce conditions before state changes. `storage` references persistent contract state, while `memory` is used for temporary data such as the constructor input. The experiment changes proposal names from `bytes32` to `string` so readable names can be supplied directly during deployment. Functions such as `vote`, `delegate`, `winningProposal`, and `winnerName` provide the main voting and result operations.

Steps

1. Open Remix IDE and access the `contracts` workspace.
2. Open `Ballot.sol` and inspect the Ballot contract, its structures, mapping, proposal array, and functions.
3. Change the proposal name type from `bytes32` to `string` and use the modified `Ballot.sol` program.
4. Compile `Ballot.sol` in the Solidity Compiler panel and verify that the Ballot contract is compiled.
5. Open Deploy & Run Transactions, select `Remix VM`, and provide the proposal names `Proposal A`, `Proposal B`, and `Proposal C`.
6. Deploy the contract and observe the deployment transaction in the transaction list.
7. Use the available contract functions to inspect the author, proposals, winner, and voting-related state.
8. Perform the required voting or delegation interaction and observe the resulting contract calls and transactions.

Code

```
1 // SPDX-License-Identifier: GPL-3.0
2
3 /*
4  * =====
5  * AIM      : Creation and Deployment of Smart Contract and
6  *            Transactions using Solidity and Remix IDE
7  * CO       : CO3, CO5, CO6
8  *
9  * Author   : Jay Kerkar
10 * Roll No: 16
11 * Batch    : A
12 *
13 * Modification (by Jay Kerkar, Roll No 16, Batch A):
14 * The original Remix Ballot.sol stores proposal names as
15 * bytes32. In this version the proposal name has been
16 * changed from bytes32 -> string, so that readable names
17 * like "Proposal A" can be passed directly at deployment.
18 * =====
19 */
20
21 pragma solidity >=0.7.0 <0.9.0;
22
23 /// @title Voting with delegation. (Modified by Jay Kerkar | Roll No: 16 | Batch: A
24 )
25 contract Ballot {
26     // Author details stored on-chain as proof of ownership
27     string public constant AUTHOR = "Jay Kerkar";
28     string public constant ROLL_NO = "16";
29     string public constant BATCH = "A";
30
31     // This declares a new complex type which will be used for variables later.
32     // It will represent a single voter.
33     struct Voter {
34         uint weight;          // weight is accumulated by delegation
35         bool voted;           // if true, that person already voted
36         address delegate;     // person delegated to
37         uint vote;            // index of the voted proposal
38     }
39
40     // This is a type for a single proposal.
41     struct Proposal {
42         string name;          // CHANGED: bytes32 -> string (Jay Kerkar, Roll 16, Batch
43         A)
44         uint voteCount;       // number of accumulated votes
45     }
46
47     address public chairperson;
48
49     // This declares a state variable that stores a 'Voter' struct for each
50     // possible address.
51     mapping(address => Voter) public voters;
52
53     // A dynamically-sized array of 'Proposal' structs.
54     Proposal[] public proposals;
```

```
55     * @dev Create a new ballot to choose one of `proposalNames`.
56     * @param proposalNames names of proposals (now as string[] instead of bytes32
    []))
57     *
58     * Example input while deploying in Remix:
59     * ["Proposal A","Proposal B","Proposal C"]
60     */
61     constructor(string[] memory proposalNames) {
62         chairperson = msg.sender;
63         voters[chairperson].weight = 1;
64
65         // For each of the provided proposal names,
66         // create a new proposal object and add it to the end of the array.
67         for (uint i = 0; i < proposalNames.length; i++) {
68             // `Proposal({...})` creates a temporary Proposal object and
69             // `proposals.push(...)` appends it to the end of `proposals`.
70             proposals.push(Proposal({
71                 name: proposalNames[i],
72                 voteCount: 0
73             }));
74         }
75     }
76
77     /**
78     * @dev Give 'voter' the right to vote on this ballot. May only be called by '
    chairperson'.
79     * @param voter address of voter
80     */
81     function giveRightToVote(address voter) public {
82         require(
83             msg.sender == chairperson,
84             "Only chairperson can give right to vote."
85         );
86         require(
87             !voters[voter].voted,
88             "The voter already voted."
89         );
90         require(voters[voter].weight == 0);
91         voters[voter].weight = 1;
92     }
93
94     /**
95     * @dev Delegate your vote to the voter 'to'.
96     * @param to address to which vote is delegated
97     */
98     function delegate(address to) public {
99         Voter storage sender = voters[msg.sender];
100         require(sender.weight != 0, "You have no right to vote");
101         require(!sender.voted, "You already voted.");
102         require(to != msg.sender, "Self-delegation is disallowed.");
103
104         // Forward the delegation as long as 'to' also delegated.
105         while (voters[to].delegate != address(0)) {
106             to = voters[to].delegate;
107
108             // We found a loop in the delegation, not allowed.
109             require(to != msg.sender, "Found loop in delegation.");
110         }
```

```
111
112     Voter storage delegate_ = voters[to];
113
114     // Voters cannot delegate to accounts that cannot vote.
115     require(delegate_.weight >= 1);
116
117     // Since 'sender' is a reference, this modifies 'voters[msg.sender]'.
118     sender.voted = true;
119     sender.delegate = to;
120
121     if (delegate_.voted) {
122         // If the delegate already voted, directly add to the number of votes.
123         proposals[delegate_.vote].voteCount += sender.weight;
124     } else {
125         // If the delegate did not vote yet, add to her weight.
126         delegate_.weight += sender.weight;
127     }
128 }
129
130 /**
131  * @dev Give your vote (including votes delegated to you) to proposal '
132  proposals[proposal].name'.
133  * @param proposal index of proposal in the proposals array
134  */
135 function vote(uint proposal) public {
136     Voter storage sender = voters[msg.sender];
137     require(sender.weight != 0, "Has no right to vote");
138     require(!sender.voted, "Already voted.");
139     sender.voted = true;
140     sender.vote = proposal;
141
142     // If 'proposal' is out of the range of the array,
143     // this will throw automatically and revert all changes.
144     proposals[proposal].voteCount += sender.weight;
145 }
146
147 /**
148  * @dev Computes the winning proposal taking all previous votes into account.
149  * @return winningProposal_ index of winning proposal in the proposals array
150  */
151 function winningProposal() public view
152     returns (uint winningProposal_)
153 {
154     uint winningVoteCount = 0;
155     for (uint p = 0; p < proposals.length; p++) {
156         if (proposals[p].voteCount > winningVoteCount) {
157             winningVoteCount = proposals[p].voteCount;
158             winningProposal_ = p;
159         }
160     }
161 }
162
163 /**
164  * @dev Calls winningProposal() function to get the index of the winner
165  contained in the proposals array
166  * and then returns the name of the winner.
167  * @return winnerName_ the name of the winner (CHANGED: bytes32 -> string)
168  */
```

```
167     function winnerName() public view
168         returns (string memory winnerName_)
169     {
170         winnerName_ = proposals[winningProposal()].name;
171     }
172 }
```

Listing 1. Ballot.sol

Output

The screenshot displays the Solidity Compiler interface. On the left, a sidebar contains various options: 'Include nightly builds' (unchecked), 'Auto compile' (unchecked), 'Hide warnings' (unchecked), 'Advanced Configurations' (expanded), 'Load AI Skills' (button), 'Compile Ballot.sol' (button), 'Security Audit' (button), 'Gas Audit' (button), 'Compile and Run script' (button), 'CONTRACT' section with 'Ballot (Ballot.sol)' selected, 'Run Remix Analysis' (button), 'Publish on IPFS' (button), 'Publish on Swarm' (button), 'Compilation Details' (button), and 'ABI' and 'Bytecode' links. The main area shows the 'Solidity Compile Details' for the 'Ballot' contract. It lists 'COMPILERINPUT' (i), 'NAME' (i), 'METADATA' (i), 'compiler' (language: Solidity), 'output', 'settings', 'sources', and 'version: 1'. The 'BYTECODE' section is expanded, showing a JSON object with 'functionDebugData' and 'abi_decode_available_length_t_array\$t_string_memory_ptr_\$dyn_memory_ptr_fromMemory'.

Fig. 1. Compiled Ballot contract details

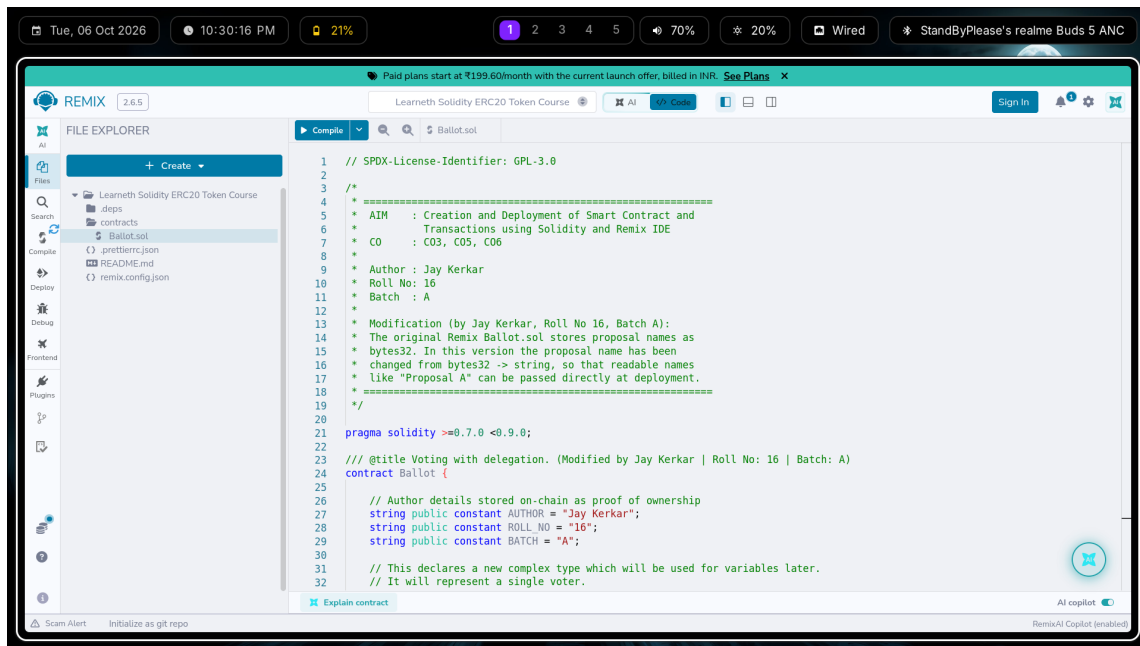


Fig. 2. Ballot source code in Remix IDE

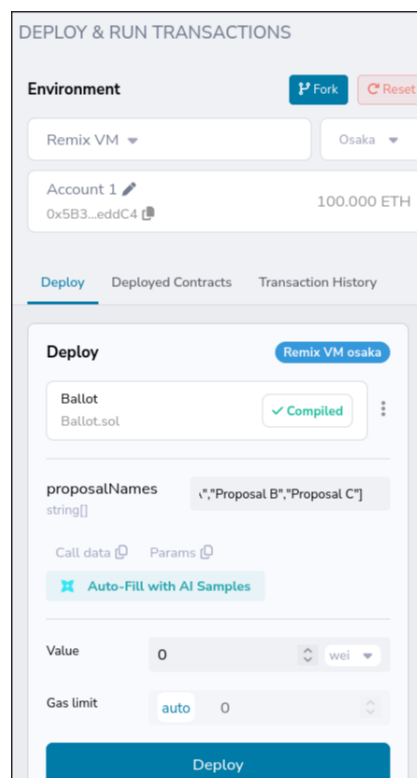


Fig. 3. Deployment form with proposal names

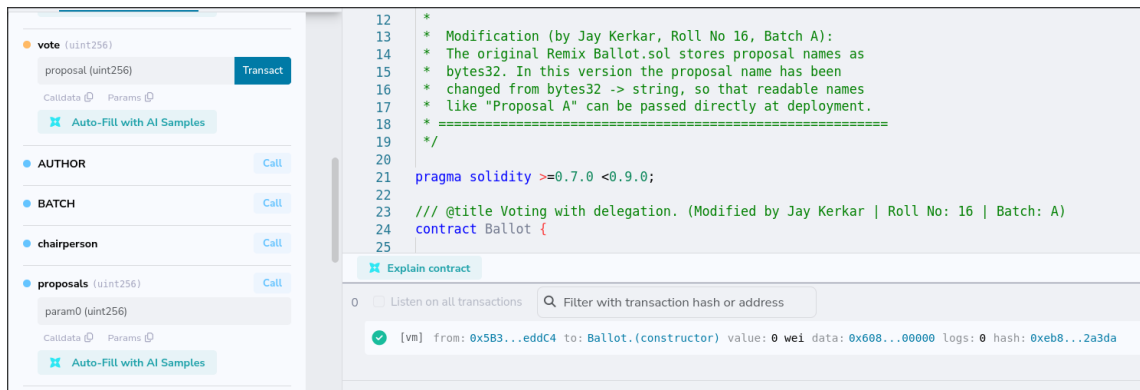


Fig. 4. Successful Ballot deployment transaction

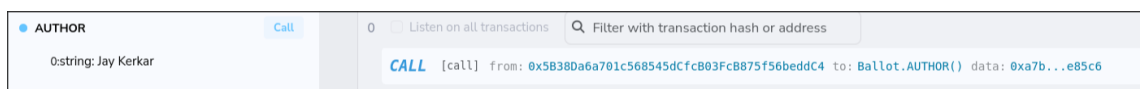


Fig. 5. Author value returned by contract call

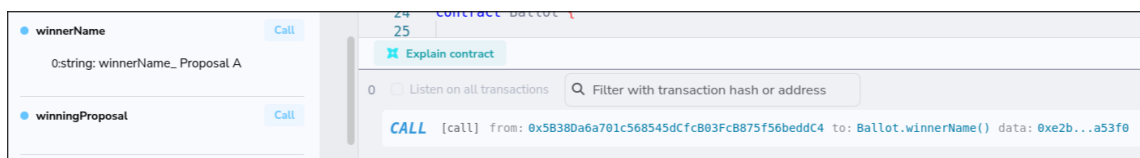


Fig. 6. Winner name returned by contract call

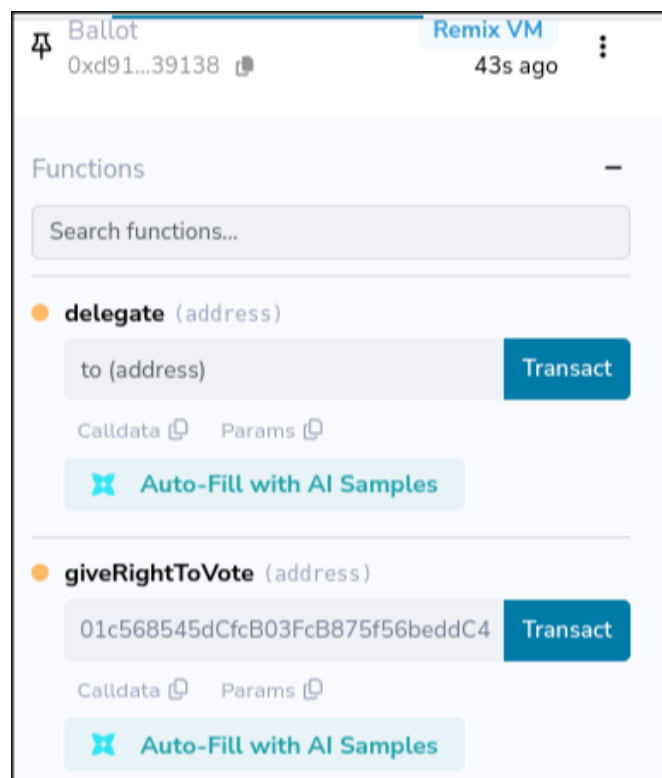


Fig. 7. Delegation and voting functions available

Conclusion

The Ballot smart contract was compiled and deployed in Remix IDE using the modified proposal-name representation. The deployment used readable proposal names and produced a successful contract transaction in the Remix VM. Contract calls displayed the stored author information and the winner name, while the deployed interface exposed voting and delegation functions. The experiment demonstrated the deployment and transaction workflow of a Solidity voting contract in Remix.